```
start, V2S_INODE_NUM(vfs_ino) * sizeof(sfs_file_entry_t), &fe,
sizeof(sfs_file_entry_t));
}
```

2) *create*, *unlink*, *lookup* (under *struct inode_operations*)—All the three functions *sfs_inode_create()*, *sfs_inode_unlink()*, *sfs_inode_lookup()* have the two common parameters (the parent's inode pointer and the directory entry for the file in consideration), and these respectively create, delete and look up an inode corresponding to a directory entry. *sfs_inode_lookup()* basically searches for the existence of the filename underneath using the already coded *sfs_lookup()* (in *real_sfs_ops.c*). If it is not found, it then invokes the generic kernel function *d_splice_alias()* to create a new inode entry in the underlying file system, for the same, and then attaches it to the directory entry *dentry*. Otherwise, it just attaches the inode from the VFS' inode cache (using the generic kernel function *d_add()*). This inode, if obtained fresh (*I_NEW*), needs to be filled in with the *sfs_lookup*-obtained file attributes. In all the above implementations and in those to come, a few basic assumptions have been made, namely:

a) Real SFS maintains mode only for the user, and that is mapped to all three of the *user*, *group* and *other* of the VFS inode.

b) Real SFS maintains only one timestamp, and that is mapped to all three—the created, modified and accessed times—of the VFS inode.

sfs_inode_create() and *sfs_inode_unlink()* correspondingly invoke the transformed *sfs_create()* and *sfs_remove()* (defined in *real_sfs_ops.c* and adapted from the earlier *browse_real_sfs* application), for respectively creating and clearing the inode entries in the underlying hardware-space file system, apart from the usual inode cache operations, using *new_inode() + insert_inode_locked()*, *d_instantiate() & inode_dec_link_count()*, instead of the earlier learnt *iget_locked()*, *d_add()*.

Apart from the permissions and file entry parameters, *sfs_create()* has an interesting transformation from user space to kernel space: *time(NULL)* to *get_seconds()*. And in both *sfs_create()* and *sfs_remove()*, the obvious user space *lseek() + write()* combo has been again replaced by the *write_to_real_sfs()*. Check out all the mentioned code pieces in the files *real_sfs.c* and *real_sfs_ops.c*.

3) The address-space operations *readpage*, *write_begin*, *writepage* and *write_end* (under *struct address_space_operations*) are basically to read and write blocks on the underlying filesystem, and are achieved using the respective generic kernel functions *mpage_readpage()*, *block_write_begin()*, *block_write_full_page()*, *generic_write_end()*. The first one is prototyped in *<linux/mpage.h>* and the remaining three in *<linux/buffer_head.h>*. Now, though these functions are generic enough, a little thought will show that the first three of these would ultimately have to do a real SFS-specific transaction with the underlying block device (the hardware partition), using the corresponding block layer APIs. And that is exactly what is achieved by the real SFS-specific function *sfs_get_block()*, which is being passed into and used by the first three functions mentioned above. Defined in *real_sfs.c*, this function is invoked to read a particular block number (*iblock*) of a file (denoted by an inode) into a buffer head (*bh_result*), optionally fetching (allocating) a new block. So for that, the block array of the corresponding real SFS inode is looked up, and then the corresponding block of the physical partition is fetched using the kernel API *map_bh()*. Note that to fetch a new block, we invoke the *sfs_get_data_block()* (defined in *real_sfs_ops.c*), which is again a transformation of the *get_data_block()* from the *browse_real_sfs* application. Also, in case of a new block allocation, the real SFS inode is also updated underneath, using *sfs_update_file_entry()*, a one-line implementation in *real_sfs_ops.c*. Code below shows the *sfs_get_block()* implementation.

```
static int sfs_get_block(struct inode *inode, sector_t iblock,
struct buffer_head *bh_result, int create)
{
    struct super_block *sb = inode->i_sb;
    sfs_info_t *info = (sfs_info_t *)(sb->s_fs_info);
    sfs_file_entry_t fe;
    sector_t phys;

    printk(KERN_INFO "sfs: sfs_get_block called for I: %ld, B:
%lld, C: %d\n", inode->i_ino, iblock, create);

    if (iblock >= SIMULA_FS_DATA_BLOCK_CNT)
    {
        return -ENOSPC;
    }
    if (sfs_get_file_entry(info, inode->i_ino, &fe) == -1)
    {
        return -EIO;
    }
    if (!fe.blocks[iblock])
    {
        if (!create)
        {
            return -EIO;
        }
        else
        {
            if ((fe.blocks[iblock] = sfs_get_data_block(info)) ==
-1)
            {
                return -ENOSPC;
            }
            if (sfs_update_file_entry(info, inode->i_ino, &fe) ==
-1)
            {
```